

Final project

Computational Finance with Python

Table of contents

General information	1
Marking and credit	1
Academic integrity	2
Submission instructions	2
1 Programming exercise	3
2 Binomial tree approximation	5
3 Finite difference method	7
4 Monte Carlo method	8

General information

Marking and credit

This project contributes 90% towards the final mark for the module, and it will be marked anonymously. In each task, marks are allocated for both for coding (style, clarity and correctness) and accuracy of numerical results, and for explanations and written work (including comments and docstrings in the code).

The marks indicated are indicative only. Partial credit will be given for partial completion of a task, for example, if you are able to do a Monte Carlo estimate with fewer paths or time steps.

It may prove impossible to examine projects containing code that does not run, or does not allow data to be changed. In such cases a mark of 0 will be recorded. It is therefore essential that all files are tested thoroughly in Colab before being submitted in the VLE.

Academic integrity

You may use and adapt any code submitted by yourself as part of this module, including your own solutions to the formative assignments. You may use and adapt all Python code provided in the VLE as part of this module, without the need for acknowledgement. **Any code not written by yourself must be acknowledged and referenced.**

This is an individual project. You **must not collaborate** with anybody else or use code that has not been provided in the VLE without acknowledgment. This includes, for example, discussing the questions with other students, actively using discussion forums or using generative artificial intelligence (such as ChatGPT). Please consult the University's [Student guidance on using AI and translation tools](#). Collusion and plagiarism detection software will be used to detect academic misconduct, and suspected offences will be investigated.

Submission instructions

Submit your work by uploading it in the VLE by the deadline. Work should be submitted as follows:

- Code (and numerical results) should be submitted in the form of Colab notebooks. Code will be tested using unseen test data: use variables to store parameters rather than using hard-wired values.
- Written text should be submitted in pdf format. Scans of handwritten text are perfectly acceptable, provided that they are legible.
- **Use a separate set of files (ipynb and/or pdf, as appropriate) for each of the four parts or the project.** "Part1.ipynb" or "part1.pdf" are examples of sensible file-names.

Late submissions will incur a penalty according to the standard rules for late submission of assessed work. It is advisable to allow enough time (at least one hour) to upload your files to avoid possible congestion in the VLE. In the unlikely event of technical problems in the VLE please email your files in a zip archive to alet.roux@york.ac.uk before the deadline.

1 Programming exercise

The following code accesses data about a US company, Pfizer, using Yahoo Finance. It then downloads 1 year of daily opening and closing prices and presents it in graphical form.

Run the code in a Colab worksheet to refresh the data.

```
import yfinance as yf
import numpy as np

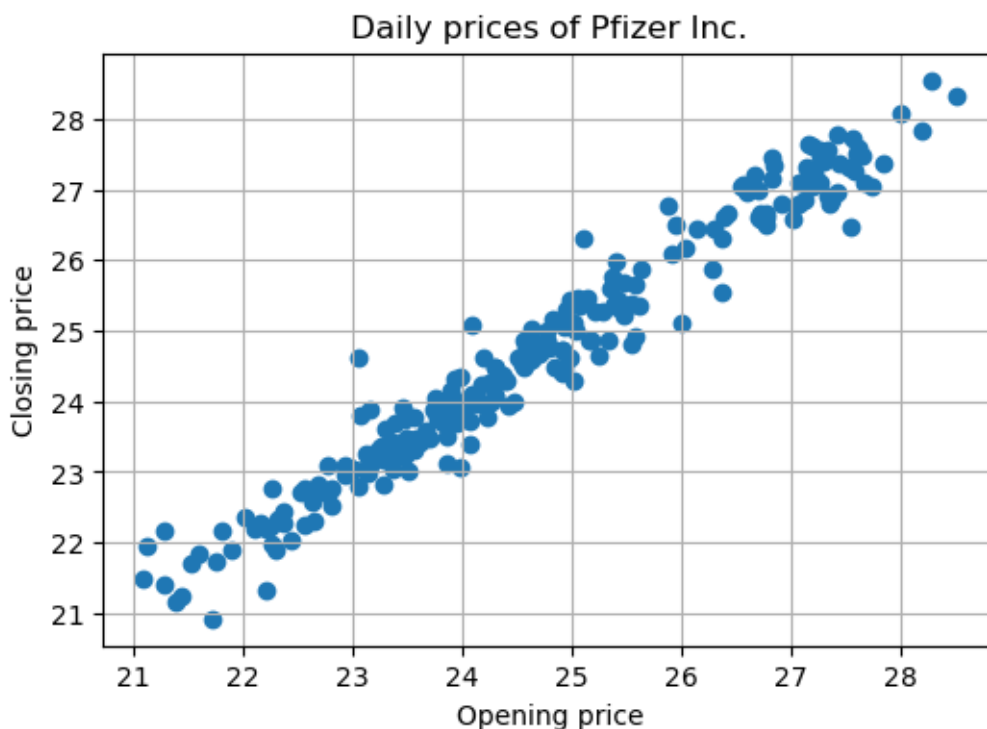
# create Ticker object which will allow data access
data = yf.Ticker("PFE")

# Download 1 year of price data
hist = data.history(period="1y")

# Store daily opening and closing prices
x = np.array(hist["Open"])
y = np.array(hist["Close"])

# Plot closing prices
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize = (6,4))
ax.set(title = f"Daily prices of {data.info['longName']}",
       ylabel = "Closing price", xlabel = "Opening price")
ax.xaxis.grid(True)
ax.yaxis.grid(True)
ax.scatter(x, y)
plt.show()
```

- ① See Arrousi (2024) for more details and usage instructions. This package is available in Google Colab, but on local installations (such as Anaconda) it may need to be installed before use—Arrousi (2024) provides instructions.



Notice from the scatterplot that there seems to be a linear relationship between the daily opening and closing prices. Taking y_i to be the daily closing price and x_i to be the daily opening price, a linear model to represent this relationship can be expressed as

$$y_i = \alpha + \beta x_i + \varepsilon_i$$

for all i , where ε_i is the error.

The least squares estimates of the parameters α and β is

$$\hat{\beta} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}, \quad \hat{\alpha} = \bar{y} - \hat{\beta} \bar{x},$$

where

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \quad \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i.$$

Exercise 1.1 (Coding task: 10% of project mark). Write a function that calculates the least squares estimates $\hat{\alpha}$ and $\hat{\beta}$.

Your function should take two arguments, the NumPy arrays x and y , and it should return the estimates as a `tuple`.

For full marks, your function should:

- Test whether x and y are the same (non-zero) length, and terminate with a suitable error message if they are not.
- Use vectorized and/or built-in NumPy methods where appropriate. Provide references for NumPy methods used that were not covered in the course materials.
- Be efficient.

Report the values of $\hat{\alpha}$ and $\hat{\beta}$ for the daily opening and closing prices of Pfizer stock downloaded above.

2 Binomial tree approximation

Consider a risky asset with stock price S that follows geometric Brownian motion under a risk neutral measure Q . In other words,

$$dS_t = rS_t dt + \sigma S_t dW_t^Q$$

or, equivalently,

$$S_t = S_0 e^{(r - \sigma^2/2)t + \sigma W_t^Q},$$

where r is the risk free rate, σ is the volatility and W^Q is a standard Brownian motion under Q .

As we have seen in the module, the continuous process S can be approximated by a binary tree. One such approach builds a tree on the interval $[0, T]$ by taking $\Delta t = T/M$, where M is the number of steps in the tree. The steps are at the equidistant times

$$t_i = i\Delta t \text{ for } i = 0, \dots, M,$$

and $S_i^{(M)}$ is the relevant approximation of S_{t_i} .

At any time t_i and over a small time interval Δt , the approximation $S_i^{(M)}$ of S_{t_i} is multiplied by u_M or d_M respectively, i.e.

$$S_{i+1}^{(M)} = \begin{cases} S_i^{(M)} u_M & \text{with probability 0.5} \\ S_i^{(M)} d_M & \text{with probability 0.5.} \end{cases}$$

The drift and volatility parameters of the continuous time process are now captured by the new parameters u_M and d_M , together with the original parameters r and σ . We take $S_0^{(M)} = S_0$ and assume that $u_M > d_M$.

Exercise 2.1 (Binomial tree approximation: 25% of project mark). Design a binary tree method that approximates the Black-Scholes model and can be used to price European options. Your work should include the following:

1. Theoretical work:

(a) Compute the conditional moments

$$\mathbb{E}^Q \left(S_{(i+1)\Delta t}^k \middle| S_{i\Delta t} = S \right)$$

in the continuous-time model and

$$\mathbb{E}^Q \left(\left(S_{i+1}^{(M)} \right)^k \middle| S_i^{(M)} = S \right)$$

in the binomial tree model, where $k = 1, 2$.

(b) Given the values of M and Δt , derive formulae for u_M and d_M in terms of r and σ by matching the first and second order conditional moments of the continuous-time model and the binomial tree model.

Hint: use the quadratic formula

$$x = \frac{1}{2a} \left(-b \pm \sqrt{b^2 - 4ac} \right)$$

for the solutions of the quadratic equation $ax^2 + bx + c = 0$.

2. State the algorithm used for approximating the price of a European-style derivative security with payoff at time T given by $f(S_T)$ by means of this tree approximation.

3. Write a Python function that uses this tree approximation to calculate the price at time 0 of a European style derivative security with payoff at time T given by $f(S_T)$. For full marks, your function should:

- Take the arguments S_0, T, r, σ, M and f , and return the price.
- Use vectorized and/or built-in NumPy methods where appropriate. Provide references for NumPy methods used that were not covered in the course materials.
- Be efficient.

4. Use your code to price a derivative with payoff

$$f(S) = \begin{cases} S - L & \text{if } S > K, \\ 0 & \text{otherwise} \end{cases}$$

and the data $S_0 = £100, T = 1.5, r = 0.05, \sigma = 0.2, M = 500, K = S_0$ and $L = £90$.

5. Write a few paragraphs (no more than 200 words) to compare this tree approximation to the one that was covered in the lectures. Which method would you recommend, and why?

3 Finite difference method

In a certain interest rate model, the price of a risk-free bond with maturity date $T > 0$ is given at time $t \in [0, T]$ by the function $F(t, r)$, where r is the value of the short-term interest rate. Under the no-arbitrage condition it satisfies

$$F(T, r) = 1$$

for all $r \geq 0$, together with the partial differential equation

$$\frac{\partial F}{\partial t}(t, r) + a(b - r) \frac{\partial F}{\partial r}(t, r) + \frac{1}{2} \sigma^2 r \frac{\partial^2 F}{\partial r^2}(t, r) - rF(t, r) = 0 \quad (1)$$

for all $t \in (0, T)$, $r \geq 0$. Here a , b and σ are given parameters.

The following facts are generally known about the function F :

- Theoretical bond prices are usually between 0 and 1, in other words, it is safe to assume that $F(t, r) \in [0, 1]$ for all $t \in [0, T]$, $r \geq 0$.
- Interest rates are usually small, in other words, it is safe to assume that $r \leq 0.5 = 50\%$ in practical applications.
- The behaviour of F when interest rates are small can be modelled as

$$\lim_{r \rightarrow 0} F(t, r) = A(t)$$

for all $t \in [0, T]$, where

$$A(t) = \left[\frac{2\gamma e^{(a+\gamma)t/2}}{(\gamma + a)(e^{\gamma t} - 1) + 2\gamma} \right]^{2ab/\sigma^2}$$

and $\gamma = \sqrt{a^2 + 2\sigma^2}$.

- The behaviour of $\frac{\partial F}{\partial r}(t, r)$ when rates are large is asymptotically similar to $-A(t)B(t)e^{-B(t)r}$, where

$$B(t) = \frac{2(e^{\gamma t} - 1)}{(\gamma + a)(e^{\gamma t} - 1) + 2\gamma}$$

and A and γ are as before. In mathematical terms this can be expressed as

$$\lim_{r \rightarrow \infty} \left(\frac{\partial F}{\partial r}(t, r) + A(t)B(t)e^{-B(t)r} \right) = 0.$$

Exercise 3.1 (Backward Euler method: 30% of project mark). Design a backward Euler finite difference scheme to approximate the solution to the partial differential equation in Equation 1. Your work should include the following:

1. Convert the final value problem into an initial value problem.
2. Propose appropriate boundary conditions for r . You should take the minimum value of r to be 0, and the maximum value to be $r_{\max}$ (to be chosen later).
3. Define a suitable grid to discretize the domain of F , and derive an iterative scheme in matrix form that would allow you to approximate the value of F at the grid points, using the boundary values.
4. Write a Python function to implement the iterative scheme. To obtain full marks, your function should:
 - Take the arguments $r_{\max}$, T , a , b , σ , as well as the number M of grid points in r and N of grid points in time-to-maturity.
 - Return the following:
 - Approximate values of F at all grid points.
 - A Boolean variable to indicate whether the method is stable or not.
 - The t values used for grid points.
 - The r values used at grid points.
 - Use sparse matrices. Provide references for SciPy methods used that were not covered in the course materials.
 - Use vectorized and/or built-in NumPy methods where appropriate. Provide references for NumPy methods used that were not covered in the course materials.
 - Be efficient.
5. Using the parameter values $a = 2$, $b = 0.05$, $\sigma = 0.2$ and $T = 1.5$, use your function to approximate $F(0, r)$ for $r \in (0, 0.15)$ and $t \in [0, T]$ and display the results graphically. Choose M and N such that the iterative scheme is stable, and choose an appropriate value of $r_{\max}$. Give details of your reasoning.

4 Monte Carlo method

A model for a stock S is given by the stochastic differential equation

$$dS_t = rS_t dt + \alpha S_0^\beta S_t e^{\gamma \sin \delta t} dW_t^Q, \quad (2)$$

where W^Q is a Brownian motion under the risk-neutral probability Q . Here $r = 0.05$, $\alpha = 0.3$, $\beta = -0.2$, $\gamma = 0.05$ and $\delta = 3\pi$ are parameters and $S_0 = £100$ is the initial stock price.

The aim of this part of the project is to study a European style derivative where the payoff $f(S_T)$ at time $T = 1$ is only paid if the stock price has not hit a barrier $L = £110$, in other words, the payoff at time $T = 1$ is

$$f_L(S, (S_t)_{t \in [0, T]}) = \begin{cases} f(S_T) & \text{if } \max_{t \in [0, T]} S_t < L, \\ 0 & \text{if } \max_{t \in [0, T]} S_t \geq L. \end{cases}$$

The price of this derivative at time 0 is then $e^{-rT} E^Q(f_L(S, (S_t)_{t \in [0, T]}))$.

Exercise 4.1 (Monte Carlo method: 35% of project mark). Implement Monte Carlo simulation to estimate the price of this derivative at time 0 under Q , as follows.

1. Write down a statement (step by step) of the Monte Carlo procedure to estimate the price of the derivative by applying the Euler scheme. For full marks, use the following techniques:
 - Transform to $Y_t = \ln S_t$ by means of the Itô formula, and then apply the Euler scheme to the process Y .
 - Antithetic sampling.
 - Control variate. Motivate your choice of control variate.
2. Implement the procedure in Python. Your code should report the Monte Carlo estimate as well as the standard error and an approximate 95% confidence interval. For full marks, your code should:
 - Implement the procedure as a Python function with suitable arguments and return values.
 - Be efficient.
 - Use vectorisation methods.
3. Use your code to report the Monte Carlo estimate of the price, the standard error and an approximate 95% confidence interval for a call option with payoff $f(S_T) = \max\{S_T - K, 0\}$ where $K = £100$. Use 1000 steps per path, and choose the number of samples such that the confidence interval is at most 0.1 in length.
4. Write a few paragraphs (no more than 300 words) to discuss whether and how trees and finite difference methods may be applied to approximate the price of the option in the previous task. Which of the three methods (Monte Carlo, tree, finite difference) would you recommend, and why?

5. Use your code to estimate the probability that the value of S is equal to or greater than L' at any point in the interval $[0, T]$, where $L = £100, £101, \dots, £120$. Display your results in both tabular and graphical format, with an approximate 95% confidence interval for each value of L' . Use 1000 steps per path, and choose the number of samples for each L' such that the confidence interval is at most 0.01 in length.

Arrousi, Ran. 2024. "Download Market Data from Yahoo! Finance's API." <https://github.com/ranaroussi/yfinance>.